

iSAQB Module: Formal Methods - Introduction

Active Group GmbH

Created: 2026-10-04 Sun 12:48

Scope

Formal methods guarantee ("verify") that a software system or component fulfills a (formal) specification.

Dijkstra's Dictum

"Program testing can be used to show the presence of bugs, but never to show their absence!"

– Edsger W. Dijkstra

Lehman's SPE classification

S-type programs — programs whose function is formally defined by and derivable from a *specification*.

P-type programs — programs for which the *problem* may be precisely formulated, but for which the solution must inevitably reflect an approximation of the real world.

E-type programs — programs that mechanize a human or societal activity and are *embedded* in the real world.

Meir Lehman: Programs, Life Cycles, and Laws of Software Evolution (1980)

Prerequisites for Formal Methods

- formal specification
- criticality for lives, livelihood, large amounts of money
- suitable development process
- applicable methods & tools
- willingness to use FM
- competency

"Executable Specification"

Stopped

Runs: 7/7 Errors: 7 Failures: 0

▼ net.serenitybdd.demos.todos.cucumber.MaintainTodos [Runner: JUnit 4] (319.868 s)

▼ Feature: Filtering todos (319.868 s)

- View **only completed items** #123 (46.975 s)
- Viewing items **by status** (45.242 s)
- Viewing items **by status** (45.210 s)
- Viewing items **by status** (45.205 s)
- Viewing items **by status** (45.199 s)
- Viewing items **by status** (45.204 s)
- Viewing items **by status** (46.798 s)

≡ Failure Trace

```
1 @cucumber
2 Feature: Filtering todos
3
4 In order to make me feel **a sense of accomplishment**
5 As a forgetful person
6 I want to be to _view all of things I have completed_
7
8 Scenario: View **only completed items** #123
9   Given that Jane has a todo list containing Buy some milk, Walk the dog
10  And she has completed the task called 'Walk the dog'
11  When she filters her list to show only Completed tasks
12  Then her todo list should contain Walk the dog
13
14 Scenario Outline: Viewing items **by status**
15   Given that Jane has a todo list containing <tasks>
16   And she has completed the task called 'Walk the dog'
17   When she filters her list to show only <filter> tasks
18   Then her todo list should contain <expected>
19
20 Examples: Example 1
21   | tasks                | filter | expected |
22   | Buy some milk, Walk the dog | Active | Buy some milk |
23   | Buy some milk, Walk the dog | Completed | Walk the dog |
24
25 Examples: Example 2
26   | tasks                | filter | expected |
27   | Buy some milk, Walk the dog | Active | Buy some milk |
28   | Buy some milk, Walk the dog | Completed | Walk the dog |
29
30 Examples: Example 3
31   | tasks                | filter | expected |
32   | Buy some milk, Walk the dog | Active | Buy some milk |
33   | Buy some milk, Walk the dog | Completed | Walk the dog |
34
```

Verification

- correctness of output
- prevention of bad behavior

Correctness of Output

- "The pacemaker pulses to reach the target frequency within one beat."
- "In particular, a request is allowed if some policy allows and no policy denies it. In the absence of applicable policies, a request is denied implicitly."
(Amazon Verified Permissions)
- "The output of the compiler adheres to the semantics of the program."
(CompCert)

Prevention of Bad Behavior

- "The program never exceeds stack size."
- "The vending machine will never be in state that prevents a product from being dispensed."
- "Sensitive data will never be exfiltrated by the software."

Exercise: Requirements vs. Formal Methods

You're writing the firmware for a heart pacemaker to fix bradycardia, i.e. compensate for the heart beating too slow. This needs to be hardware- and power-efficient.

What kinds of requirements might be amenable to the use of formal methods?
What could specifications for those requirements look like?

Exercise: Specification

Consider two sub-problems of a bradycardia pacemaker:

1. The pacemaker needs to provide stimuli at the right moments, bringing the heart rate to the target rate, as evenly as possible.
2. The pacemaker needs to provide electric pulses of an appropriate magnitude, i.e. not stronger than required to make the heart contract.
 - There are three magnitude levels, L, M, and H.
 - The heart can't take more than 3 pulses at H.
 - If the heart doesn't start beating at L and M, the pacemaker should pulse at H within 4 beats.

What would appropriate specifications for these two sub-problems look like?

Verification

- **specification** of properties via logic
- **mechanization** of specification
- mechanized, formal **proof** of properties

Different Kinds of Components

- functions: "Is the output right for the input?"
- data types: "Are the invariants for the type preserved?"
- algorithms: "Does it perform the right steps?"
- systems: "Is the behavior as expected?"

Different Qualities

- functionality: relationship between input and output
- performance efficiency: e.g. worst-case execution time
- reliability: e.g. free from crashes
- security: e.g. guaranteed security checks
- safety: e.g. impossibility of harmful behavior

The Problem

- infinite inputs (& outputs)
- infinite states

Approaches

- explicit proof, operating on code
- abstraction to reach finiteness,
then symbolic execution / model checking

Tools

- Model Checking
- Property Verification
 - SMT Solving (unassisted)
 - Proof Assistant
- Abstract Interpretation

Model Checking

- state-based systems
- "try all possible states"
- finite state space
- fully automatic

Tools:

- **Alloy** (invariants of relational specifications)
- **TLA+** (correctness of concurrent algorithms)

SMT Solver

- properties in first-order logic
- implicit mathematical proof
- arbitrary code
- fully automatic

Tools:

- **Z3** (stand-alone SMT solver, used from many other tools)
- **Liquid Haskell**
- **VeriFast** (C/Java/Rust)
- **Dafny** (generates C#/Java/JavaScript/Go/Python)

Proof Assistant

- properties in first-order logic
- explicit mathematical proof
- arbitrary code
- requires guidance

Tools:

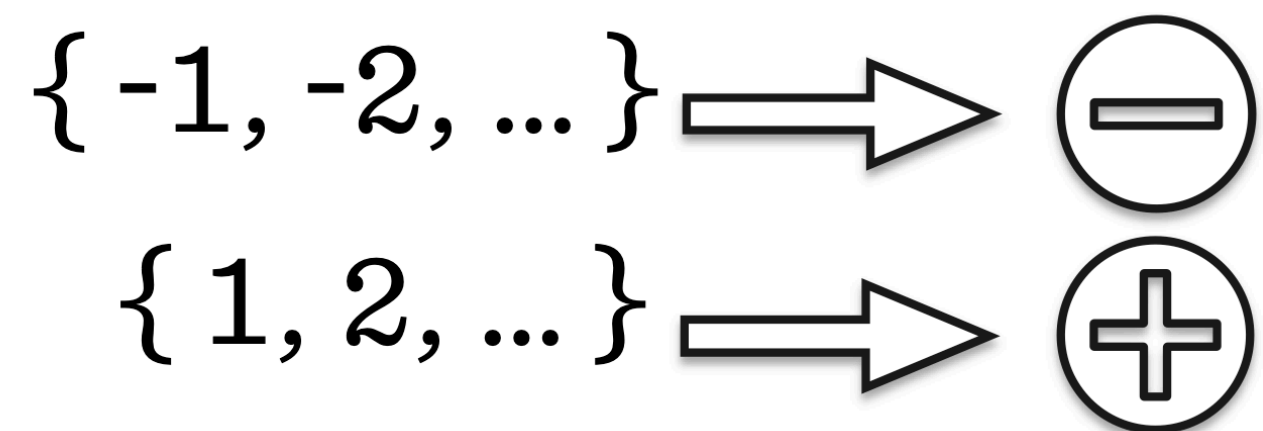
- Rocq (applied to CompCert)
- Isabelle (applied to seL4)
- ACL2 ("Applicative Common Lisp", applied to Hardware Design)
- Lean

Abstract Interpretation

- static analysis
- run abstract version of program, but finite domains
- arbitrary code
- abstraction required, but usually automatic

Tools:

- **Frama-C** (C)
- **AbsInt products** (C/C++)
- **Coverity** (C/C++/Java)
- **Infer** (C/C++/Java)



Choice

- Run-time-observable qualities
⇒ abstract interpretation
- Functional specification:
 - Small state/input space or abstraction possible?
⇒ model checking
 - Large state/input space
⇒ SMT solver/proof assistant

QuickCheck / Property-Based Testing

- types + properties => automatic test generation
- properties as functions or abstract machines
- semi-automatic - needs generators
- available for many languages

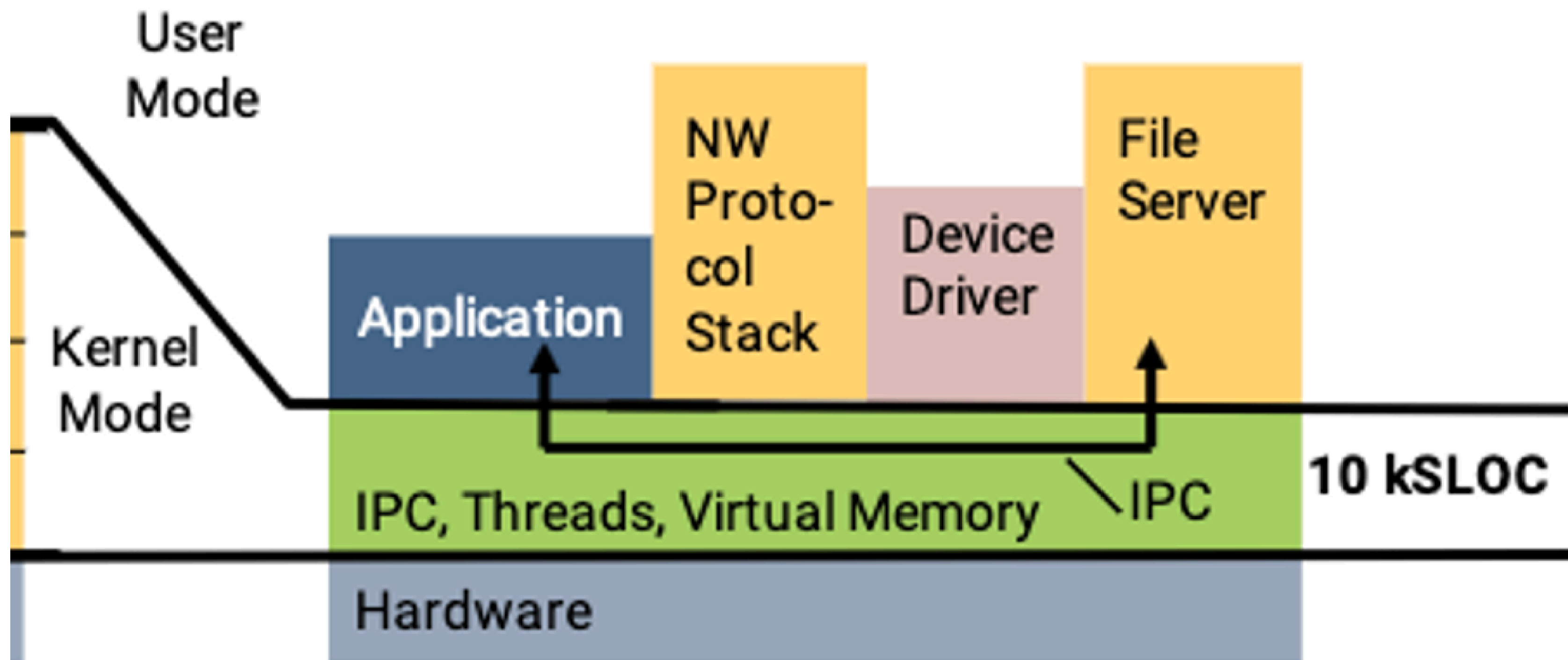
Exercise: Ensuring Correctness

Return to the two pacemaker problems. What are appropriate methods to ensure correctness for software implementing solutions?

Formal Methods for Architecture Evaluation

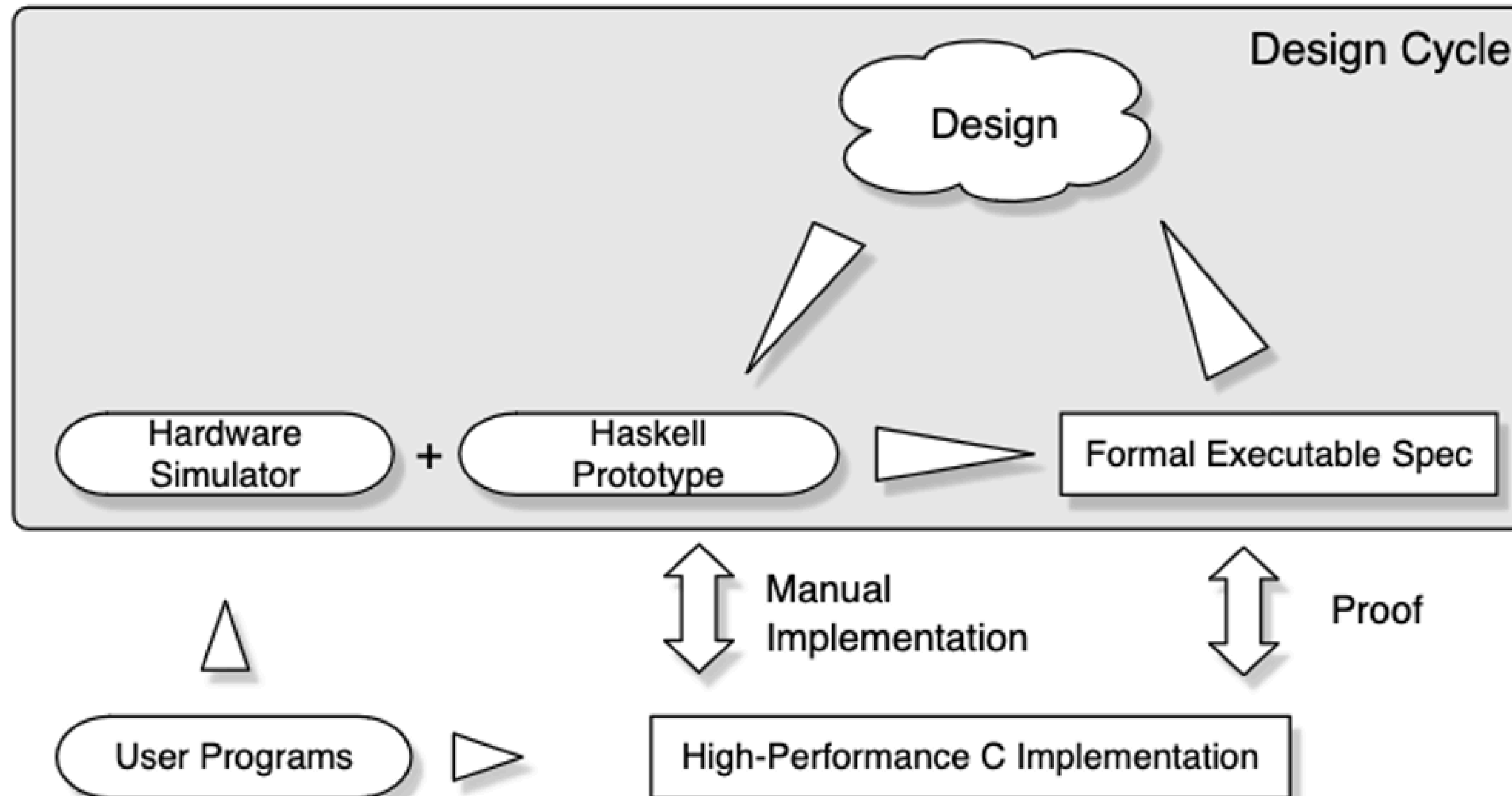
- use static analysis to track resource usage
- identify deviations from coding guidelines (such as MISRA C)
- find data races, overflows, undefined behavior

Showcase: seL4



Gernot Heiser: The seL4 Microkernel - an Introduction

seL4: Development Process



Klein et al: seL4: Formal Verification of an OS Kernel

seL4: Abstract Specification

```
schedule  $\equiv$  do  
  threads  $\leftarrow$  all_active_tcbcs;  
  thread  $\leftarrow$  select threads;  
  switch_to_thread thread  
od OR switch_to_idle_thread
```

Klein et al: seL4: Formal Verification of an OS Kernel

seL4: Executable Specification

```
schedule = do
  action <- getSchedulerAction
  case action of
    ChooseNewThread -> do
      chooseThread
      setSchedulerAction ResumeCurrentThread
    ...
chooseThread = do
  r <- findM chooseThread' (reverse [minBound .. maxBound])
  when (r == Nothing) $ switchToIdleThread
chooseThread' prio = do
  q <- getQueue prio
  liftM isJust $ findM chooseThread'' q
chooseThread'' thread = do
  runnable <- isRunnable thread
  if not runnable then do
    tcbSchedDequeue thread
    return False
  else do
    switchToThread thread
    return True
```

Klein et al: seL4: Formal Verification of an OS Kernel

seL4: Implementation

```
void setPriority(tcb_t *tptr, prio_t prio) {
    prio_t oldprio;
    if(thread_state_get_tcbQueued(tptr->tcbState)) {
        oldprio = tptr->tcbPriority;
        ksReadyQueues[oldprio] =
            tcbSchedDequeue(tptr, ksReadyQueues[oldprio]);
        if(isRunnable(tptr)) {
            ksReadyQueues[prio] =
                tcbSchedEnqueue(tptr, ksReadyQueues[prio]);
        }
    }
    else {
        thread_state_ptr_set_tcbQueued(&tptr->tcbState,
                                       false);
    }
    tptr->tcbPriority = prio;
}
```

Klein et al: seL4: Formal Verification of an OS Kernel

Refinement

Abstract Specification

→ Executable Specification

→ Implementation

seL4: Verification Effort

	Artefact	Effort (py)	Total (py)
Kernel Development	Haskell prototype	2.0	2.2
	C implementation	0.2	
Correctness Proof	Generic framework and tools	9.0	20.5
	Abstract specification	0.3	
	Executable specification	0.2	
	Refinement $\mathcal{M}_A \leftrightarrow \mathcal{M}_E$	8.0	
	Refinement $\mathcal{M}_E \leftrightarrow \mathcal{M}_C$	3.0	
Binary Verification	Verified Binary	2.0	2.0

Stevens: An Overview of the Verification of the seL4 Microkernel

System Design with Formal Methods

- verify directly
- generate from verified code
- prove equivalent to verified code

Exemplary Tools for This Training

- QuickCheck (property-based testing)
- Forge (model checking)

Pick from:

- Liquid Haskell or VeriFast (SMT solving)
- Isabelle or Agda (proof checking)

Structure

- Introduction to Logic
- Specifications as Code

Optional:

- SMT, then VeriFast
- SMT, then Liquid Haskell
- Proof Assistants